Getting Started with Python - A Beginner's Guide

LS100 — Module 00B · Python Fundamentals

Last updated: 2026-07-02

Authored by **Souvik Mandal, Ph.D.**

Project Leader & Instructor, Computational Behavioral Sciences, LS100, FAS, Harvard University | Linkedin ID: souvik-mandal-phd

Published Jul 02, 2026

1. Introduction: What is Python?

Python is an open-source, high-level programming language known for its simplicity and versatility. It is a **general-purpose** language, meaning you can use it for almost any kind of programming – from developing software and web applications to data operations, scientific computing, to flight control and industrial applications. First released in 1991, Python has become one of the most popular and recommended languages for beginners and experts alike. Its syntax is designed to be highly readable (often resembling plain English), which makes it easy to learn and **beginner-friendly**. Python's popularity is backed by a large, active community and a rich ecosystem of libraries for various domains like data science, machine learning, computer vision, automation, software & web development, etc. Most, if not all, top tech companies and research labs use Python in their technology stacks, and Python consistently ranks among the most-loved and in-demand programming technologies. In short, pick up your favorite thing that can be benefitted from computing / coding / programming, learn Python from that perspective, and the Python skills you will develop can open the doors to many other areas of computing.

2. Using Python: From Simple to Advanced Workflows

When starting out, there are multiple ways to **set up and work with Python**, each with its own pros and cons. Below we examine several approaches, from the simplest (no installation) to more complex development workflows:

2.a. Online (Cloud) Python Notebook Environments

One of the easiest ways to start coding in Python is through Notebooks on **online Python environments** like **Google Colab,** and **Kaggle**. We can write and run Python code in these notebooks through web browsers, just like any other website, with no installation on our computer.

- **Pros:** No need to install anything locally – you can start coding immediately in a browser. Colab also provides free access to high-end computing resources (like GPUs/TPUs) and comes with many libraries pre-installed. It's easy to share notebooks and collaborate, since everything is stored online in your Google

Drive. You also don't have to worry about managing packages or dependencies initially, as the environment is pre-configured.
- **Cons:** Requires a stable internet connection to use. You are limited by the cloud environment's restrictions (runtime limits, certain filesystems). Data privacy could be a concern if you're uploading sensitive data to the cloud. Additionally, you have less control over the environment (for example, you might not be able to install arbitrary system tools). Performance can be limited for very large datasets since you depend on remote servers and network speed.

When to use: Online notebooks are great for initial learning, short projects, or when you can't install software on your device. Educators often use them so learners can focus on coding rather than environment setup. If you have internet access and want to start right away, this is the simplest route.

2.b. Standard Installation on Your Computer

Another approach is to **install Python on your computer** (computer is also called "machine" in this context) and run code *locally* (which means Python running on your computer). The official Python distribution can be downloaded from Python's official website (https://www.python.org/downloads) for Windows, macOS, or Linux. This approach gives you a persistent Python setup on your machine.

- **Pros:** Full control over your environment, and you can work offline. The official installer comes with the **Python interpreter** and the **IDLE** development environment (a simple code editor) included. You can run Python programs from your command line or terminal, and install additional libraries as needed using Python's package manager (pip). Running code locally can be faster for data stored on your computer, and you're not constrained by cloud session limits.
- **Cons:** Requires you to download and **install** software, which might be challenging if you don't have admin rights or experience with installations. You also need to manage updates and package installations yourself. On Windows, a common hurdle is ensuring the *PATH* is set correctly (so that the `python` command is recognized in the terminal) – fortunately, the installer has an option "Add Python to PATH" to simplify this. Another consideration is that you might encounter version conflicts or need to set up a **virtual environment** to isolate project dependencies as you install more packages (this is more of an issue in advanced use, but worth noting).

When to use: Installing Python locally is recommended if you plan to do substantial work on your own machine, want the ability to save files locally, or if you have offline requirements. It's the next step once you outgrow the simplicity of online notebooks and need more control or want to build larger projects.

2.c. Installing Locally using a Python Distribution (Anaconda for Data Science)

For learners interested in data science or scientific computing, an **all-in-one distribution** like **Anaconda** is a popular choice. Anaconda bundles Python with a collection of commonly used libraries (NumPy, Pandas, matplotlib, etc.) and includes tools like **Jupyter Notebook** and **JupyterLab** by default.

- **Pros:** One installation gives you a ready-to-use environment for data analysis and machine learning. Many packages come pre-installed, so you avoid dealing with dependency issues initially. It also provides a user-friendly launcher (Anaconda Navigator) to open notebooks or other tools without command-line usage. This can simplify setup for beginners focusing on data tasks.
- **Cons:** The download size is large (hundreds of MBs) because of all the bundled packages. It may install some packages you don't need, using significant disk space. Anaconda manages packages with its own tool (`conda`), which means you might need to learn a slightly different workflow than using pip. Mixing `conda` and `pip` can sometimes lead to environment conflicts, so it requires a bit of care. Additionally, because it's a separate distribution, it might not always have the very latest Python version or package releases (though it's usually up to date).

When to use: If your primary goal is data science or you want a hassle-free way to get a scientific Python stack, Anaconda is a good choice. It's often used in academic settings for its convenience. However, for general programming or if you prefer a lean setup, the standard Python installation plus manual package installs might be preferable.

2.d. Integrated Development Environments (IDEs) and Code Editors

As you progress, you may want to use a more powerful code editor or **IDE** (Integrated Development Environment) for writing Python code. Free IDEs like Visual Studio Code (VS Code), PyCharm, or Spyder offer rich features like syntax highlighting, auto-completion, debugging, and project management. Most of these IDEs offer their Pro version for free to students and educators.

- **Pros:** An IDE can greatly enhance productivity for larger projects. Features like IntelliSense (auto-complete suggestions), real-time error highlighting (linting), and integrated debugging help catch mistakes and understand code faster. Many IDEs also integrate with version control (Git) and have plugins for Python that make tasks like testing or refactoring easier. VS Code, for instance, is lightweight yet powerful – with the official Python extension, it becomes a versatile Python IDE.
- **Cons:** There is a learning curve to using an IDE effectively. Beginners might find the interface and options overwhelming at first. You also have to install and configure the IDE in addition to Python, which is an extra step. Some IDEs (like PyCharm) are heavier on system resources, which might be an issue on older or low-spec computers. It's easy to stick with simpler editors at the very beginning and then transition to an IDE when projects become more complex.

When to use: If you start working on multi-file projects, need to debug code, or just prefer a more feature-rich environment, an IDE is appropriate. Many beginners start with IDLE or simple text editors and then move to VS Code or PyCharm as they become more comfortable. Visual Studio Code in particular is a friendly stepping stone – you can use it just as a text editor initially, and gradually make use of its IDE features as needed.

2.e. Advanced Workflows: Virtual Environments and Cloud Deployment

For completeness, it's worth mentioning what more advanced Python workflows look like (even if it's beyond the initial learning phase):

- **Virtual Environments:** As you install more packages, you'll encounter situations where different projects require different package versions. **Virtual environments** are isolated spaces for project-specific dependencies. Tools like `python -m venv` (built-in) or `conda ``env` or `pyenv` help manage this. For example, you might create a virtual environment for a web app project so that its dependencies don't conflict with those of a data science project. Using virtual environments is considered one of the best practices in Python development, you may not need this immediately if you're just starting and using one environment for everything.
- **Cloud Deployment:** Eventually, you may want to turn your Python code into an application that others can use (for example, a web service or a data dashboard). Python supports this through web frameworks like **Flask** or **Django**, and you can deploy apps to cloud platforms (AWS, Heroku, etc.) or as simple cloud functions. This is an advanced step, but it's the direction many projects go. Thanks to Python's flexibility, the same code you prototype in a notebook can often be integrated into a web application or cloud workflow. (For instance, Python is commonly used in web development and can be the foundation of full software products. Mastering the basics will prepare you to explore these deployment tools when you're ready.

In summary, **beginners should start with the approach that gets them coding as easily as possible** – an online notebook or a basic local install – and gradually adopt more sophisticated tools as needed. Python's strength is that it scales with you: you can start with a single notebook cell and, down the road, end up building a complex cloud-deployed application using the same core language.

3. Installing Python On macOS (Apple M Silicon)

If you are using a Mac with an Apple Silicon chip (M1, M2, etc.), installing Python is straightforward. Newer versions of macOS (since macOS Catalina) do *not* come with a usable Python 3 installation by default [1], so you will need to install Python 3 yourself. There are a couple of methods:

Method 1: Install via Homebrew (Recommended for Developers). If you're comfortable with the Terminal, you can use Homebrew, a package manager for macOS, to get Python. First, install Homebrew (if not already installed) by running the official install script in Terminal [1]. Then install Python by typing:

```
brew install python3
```

This will fetch the latest Python 3 version. Homebrew makes it easy to update Python later (`brew upgrade python`) and manages the installation for you.

Method 2: Download the Official Python Installer. The Python Software Foundation provides macOS installers on the official Python website. Visit the downloads page and click the "Download Python 3.XX.X" button for your operating system. Make sure to choose the **universal installer** package, which supports both Intel and Apple Silicon Macs (modern Python installers include native support for Apple's Silicon chips). Download the `.pkg` file and run it, then follow the prompts through the installation wizard (the default settings are usually fine). You may be asked for your Mac login password to approve the install.

> **Apple Silicon Note:**
>
> The universal installer will install an optimized version of Python for your M1/M2 Mac. In the past, there were workarounds using Rosetta or specific builds, but nowadays the official Python 3.11+ installer works natively on Apple Silicon Macs [24], so you get full performance out of Python.

After installation completes, you can verify it was successful. One way is to open the **Terminal** (find it in Applications > Utilities) and type `python3 --version` (or `python3` to enter the interactive shell) [26]. Another way is to use **IDLE**, the Integrated Development Environment that comes with Python. Find IDLE in your Applications (it might be in a folder called "Python 3.x"). Launch IDLE, and it will open a window with the Python prompt `>>>`. You can try typing a simple command like `print("Hello, Python!")` and see the output. If everything is installed correctly, IDLE will show the Python shell waiting for your commands.

> **Tip**
>
> By default, the `python` command on macOS might still refer to Python 2 (on some older systems). It's safer to use `python3` to run Python 3. Also, consider installing a code editor or IDE (like VS Code, mentioned earlier) for a better coding experience once Python is set up.

Method 3: Install via Anaconda Distribution.
Anaconda is a popular all-in-one Python distribution designed for **data science and machine learning**. It comes with Python plus hundreds of commonly used libraries and tools already installed, including **Jupyter Notebook** and **JupyterLab**.

- Download the **Anaconda installer for macOS (Apple Silicon)** from the Anaconda website.
- Run the installer and follow the instructions (defaults are fine).
- After installation, you can open **Anaconda Navigator**, a graphical interface for launching Jupyter, Spyder, or managing environments.

Pros:

- Beginner-friendly: Python, Jupyter, and common libraries are included out of the box.

- Optimized for **data science and research workflows**, making it a good fit for this course.
- Works natively on M1/M2 chips with the latest builds.
- Let you manage multiple isolated environments easily with conda.

Cons:

- Large download size (several hundred MB), taking more disk space by default.

4. Installing Python on Windows

Installing Python on Windows is also easy, with two main options available. You can either use the **Microsoft Store app** or install Python from the **official website**. We'll cover both:

Option 1: From Microsoft Store (Windows 10/11). Microsoft provides Python via the Windows Store, which is a quick way to get Python 3 set up without dealing with PATH settings or admin permissions. Simply open the Microsoft Store, search for **Python 3.** (look for the one published by the Python Software Foundation), and click **Get/Install**. This will install Python for your user account. The Store installation automatically configures your PATH, so you can immediately use the `python` command in PowerShell or Command Prompt. It also will automatically update Python when new versions are released. This method is recommended for beginners by Microsoft's documentation because it's simple and doesn't require extra configuration. After installation, you can verify by opening a new PowerShell (or Command Prompt) and typing `python --version` to see the installed version. You should also find **IDLE** in your Start menu (e.g. "IDLE (Python 3.x)") which you can launch to bring up the Python shell.

Option 2: Using Winget Winget (Windows Package Manager) is a command-line tool built into modern versions of Windows that lets you install apps quickly from Terminal or PowerShell. Open **PowerShell** and run `winget install Python.Python.3` to install the latest Python 3 release. Winget handles downloading and installing Python for you, and this method is especially useful for learners who prefer terminal-based setup. For official step-by-step instructions, see Microsoft's guide: Install Python with Winget.

Option 3: Using the Official Installer from Python.org Go to python.org/downloads and download the latest Windows installer (choose the 64-bit installer for modern PCs). Run the installer `.exe` file. **Important:** On the installer's first screen, check the box that says **"Add Python to PATH"** (and optionally "Install launcher for all users") before clicking "Install Now". This ensures that the Python executable is added to your system PATH, which means you can use the `python` command from any terminal window. The installer will then copy Python onto your system. After a few minutes, you should see a success message.

During the Python installation on Windows, make sure to check the option to Add Python to PATH (as shown above). This will let you run `python` *from the command line without extra steps.*

Once installed, verify it by opening the **Command Prompt** (cmd.exe) and running `python --version`, which should display the Python 3 version. You can also launch the **IDLE** app (search for "IDLE" in Start) – if it opens and shows a Python shell window, the installation was successful. From the command line, the `python` (or `python3`) command will start the interactive interpreter, where you'll see the `>>>` prompt. Type `exit()` or press Ctrl-Z then Enter to quit the interactive mode.

Option 4: Anaconda Distribution. If you prefer using Anaconda on Windows, download the Anaconda installer from anaconda.com (it's a large .exe file). Running it will install Python and a suite of tools. Anaconda has its own Start menu entry (Anaconda Prompt, Jupyter Notebook, etc.). This is an alternative to the above, mainly for those focusing on scientific computing.

After installing via any method, you are ready to write and run Python code on Windows. If you plan to use a code editor like VS Code, be sure to install it and the Python extension as mentioned earlier. Visual Studio Code can detect your Python installation automatically, especially if Python is in the PATH.

> **Note**
>
> For web development on Windows, you might encounter advice to use the Windows Subsystem for Linux (WSL) to run Python in a Linux environment. That's an advanced setup for specific needs (e.g. if deploying on Linux servers), and not necessary for beginners. The native Windows Python will work for learning purposes and most tasks.

5. Working with Python: Running Scripts and Notebooks

With Python installed (or an online environment ready), you can now start **running Python code**. There are a few common ways to use Python interactively or run programs:

- **Interactive mode (REPL):** If you type `python` in a terminal/command prompt, you enter the interactive mode, also known as the REPL (Read-Eval-Print Loop) or simply the Python shell. You'll see a `>>>` prompt. You can type Python statements one by one and see results immediately. This is useful for quick experiments or calculations. Both IDLE and the terminal `python` command provide this interactive interpreter[35]. For example, entering `2 + 2` will instantly output `4`. To exit the interactive mode, use `exit()`.
- **Running scripts:** You can write Python code in a text file with a `.py` extension (this is called a *script*) and run it as a whole. For instance, you might create a file `hello.py` containing `print("Hello")`. Running this script can be done by executing `python hello.py` in a terminal. If using an IDE or code editor, there

are typically run commands (for example, in VS Code you can press the Run button or F5 to execute the script). This mode is better for saving and re-running programs, and it's how larger applications are developed.

- **Jupyter Notebooks:** A Jupyter Notebook is an interactive web-based interface that lets you mix code, text, and visualizations. It's extremely popular in education and data science. You can start a local Jupyter Notebook server by installing the notebook package (`pip install notebook`) and running `jupyter notebook`. This will open a browser interface where you can create notebooks. In a notebook, you write code in cells and execute them to see output below the cell. Notebooks are great for step-by-step exploration and keeping notes alongside code. If you installed Anaconda, Jupyter is already included (you can launch it from the Anaconda Navigator or the Start menu on Windows). If you prefer not to install anything, remember you can use **Google Colab**, which is essentially a hosted Jupyter Notebook provided by Google, accessible at colab.research.google.com.

6. Installing Jupyter Notebook (for Method 1 & 2 users)

If you installed Python via **Homebrew** or the **official .pkg installer**, Jupyter Notebook is **not included by default**. You need to install it separately. Follow the steps below:

1. Open **Terminal**.
2. Upgrade pip (optional but recommended):

```
python3 -m pip install --upgrade pip
```

3. Install the Jupyter Notebook package:

```
python3 -m pip install notebook
```

4. To launch Jupyter, navigate to your project folder in Terminal and run:

```
jupyter notebook
```

This will open a browser window at http://localhost:8888, where you can create and run notebooks.

> **Note**
>
> If you used **Anaconda (Method 3)**, Jupyter is already installed — you can launch it directly from Anaconda Navigator or by typing jupyter notebook in Terminal.

When to use notebooks vs scripts: For learning and experimentation, notebooks are fantastic because you can run pieces of code in isolation and see results (graphs, data tables, etc.) immediately. Our guide will next have you use a Jupyter Notebook to practice Python concepts. On the other hand, if you're writing a program that will be used as an application or needs to be packaged, you'd typically organize it into script files or modules. It's common to use notebooks for exploration and then move code into `.py` files for deployment or larger projects.

Now that your environment is set up and you know how to run Python, let's start our first exploration with Python.

7. Hands-On Practice: Using a Jupyter Notebook to Explore Python

At this point, you have Python set up and know the basic concepts. Now, it is time to play with code in a **Jupyter Notebook**. Please visit the module 00B on the course website and start practicing Python with the notebooks in numeric order. You can either use an online notebook environments like Google Colab or run the notebook locally.

8. Conclusion: From Basics to Building Full Pipelines

Congratulations on making it through this introduction to Python! You've learned what Python is, set up an environment, and covered fundamental programming concepts from variables and functions to loops and classes. At this stage, you should be able to write simple programs and use notebooks to experiment with code. You even saw how to bring in external libraries to work with real-world data like audio and video.

Learning programming is a bit like learning a language – at first, you learn words and grammar (we covered the "words" like variables, and "grammar" like syntax/loops/functions). The next step is to practice by actually "speaking" – writing programs to solve problems or automate tasks. Start with small projects or exercises: for example, write a program to convert units (temperature, currency), or a simple number guessing game, or analyze a small dataset.

As you become comfortable with the basics, you can explore more advanced topics or domains:

- **Data Science/Analysis:** Learn libraries such as pandas, NumPy, matplotlib, and explore how to manipulate datasets and create visualizations.
- **Automation/Scripting:** Use Python to automate tasks on your computer – for example, rename files in bulk, scrape data from websites (with libraries like BeautifulSoup), or send automated emails.
- **Machine Learning/AI:** Explore libraries like scikit-learn or TensorFlow once you have a handle on Python – the strong foundation in Python will allow you to grasp these libraries faster.
- **Web Development:** Try a web framework like Flask or Django to build a simple web app. Python's flexibility means the same fundamentals you learned apply here – you'll just be writing functions that respond to web requests instead of a loop in a notebook. Python being used in web development is one of the reasons for its broad adoption.
- **Developing and Deploying Apps:** Eventually, you might combine your Python knowledge with frameworks and cloud services to create deployable applications. Python's simplicity and the vast array of tools available mean that with time, you could write a Python program and deploy it as a cloud service or

application that others can use. (For instance, many cloud services support Python natively, and you can deploy Python web apps to platforms like Heroku, AWS, etc., turning your code into a live service.)

Keep in mind that Python was created to be an easy-to-understand language, and it has a very supportive community. There are countless resources and forums (like Stack Overflow, Reddit's r/learnpython, etc.) where you can seek help and advice.

With the guide and the hands-on practice in the notebook, you're well on your way. Happy coding, and welcome to the world of Python programming! As you continue, you'll discover more *cool reasons to learn and use Python*, including the ability to quickly turn ideas into working code. All the best for your journey to using Python for your research. Python's versatility and power will be your ally every step of the way. Enjoy!